

Gestor de Recursos Distribuido HPC

R-322 — Sistemas Operativos I

Juan Ignacio Gorgo
Camila Sarasúa
Lucio Oviedo Dell’Orefice
Santiago Salvático

23 de junio de 2026

Índice

1. Descripción General del Proyecto	3
2. Arquitectura del Componente Erlang	3
2.1. Descripción General de los Módulos	3
2.2. Arquitectura de Procesos	3
2.3. Convención del Entorno de Pruebas	4
2.4. Secuencia de Inicio	4
2.5. Secuencia de Inicio del Agente C	4
3. Protocolo de Comunicación	5
3.1. Protocolo Inter-Agente (C a C)	5
3.2. Interfaz Local (Erlang a Agente C)	5
3.3. Diagrama de Secuencia: Ciclo de Vida de un Trabajo	6
4. Estrategia de Prevención de Deadlocks	7
4.1. Descripción del Problema	7
4.2. Estrategia: Ordenamiento Global de Recursos	7
4.3. Por Qué Funciona: Ejemplo Paso a Paso	7
5. Registro de Eventos (Event Logger)	8
5.1. Diseño	8
5.2. Registro de Log (Log Record)	8
5.3. Ejemplo de Salida	8
5.4. Justificación del Diseño	8
6. Contribuciones del Equipo	9
7. Desafíos de Implementación	9
7.1. Identificación de Agentes	9
7.2. Propiedad de Recursos Remotos	9
7.3. Interoperabilidad del Parser entre C y Erlang	9
7.4. Tráfico de Descubrimiento UDP	10

8. Problemas Encontrados y Soluciones	10
8.1. Colisión de Job ID entre Nodos en la Misma IP	10
8.2. Cierre prematuro del socket	10
8.3. Registro del ID del proceso de registro (Logger)	11
8.4. Bloqueo de lectura TCP vs. paso de mensajes en Erlang	11

1. Descripción General del Proyecto

Este informe describe el diseño e implementación de un gestor de recursos distribuido para un clúster HPC simulado. Cada nodo consta de dos componentes cooperativos: un **Agente Gestor de Recursos** escrito en C, y un **Planificador de Trabajos (Job Scheduler)** escrito en Erlang. Los componentes se comunican a través de TCP en localhost, mientras que los agentes C se comunican entre sí a través de la red utilizando un protocolo ASCII común.

El lado de Erlang es responsable de la generación de trabajos, la orquestación de solicitudes de recursos, la prevención de interbloqueos (deadlocks) y el registro de eventos. El lado de C gestiona los recursos locales, maneja conexiones concurrentes a través de `epoll` e implementa el descubrimiento de nodos basado en UDP.

2. Arquitectura del Componente Erlang

2.1. Descripción General de los Módulos

El componente Erlang está estructurado en cuatro módulos con responsabilidades claramente separadas:

Módulo	Responsabilidad
<code>header.hrl</code>	Macros compartidas, registros y constantes de ordenamiento de recursos
<code>erlang_c_bridge</code>	Gestión de conexiones TCP con el agente C local
<code>job_scheduler</code>	Generación de trabajos, lógica de planificación y gestión de estado
<code>event_logger</code>	Registro asíncrono de eventos en un archivo

2.2. Arquitectura de Procesos

El sistema consta de dos componentes principales por nodo: un agente C y un planificador Erlang. El lado de Erlang utiliza el modelo de actores: cada unidad lógica se ejecuta como un proceso independiente que se comunica exclusivamente a través del paso de mensajes, sin memoria compartida. El agente C maneja toda la E/S de red utilizando `epoll`, delegando la lógica de recursos a módulos internos conectados a través de una interfaz bien definida.

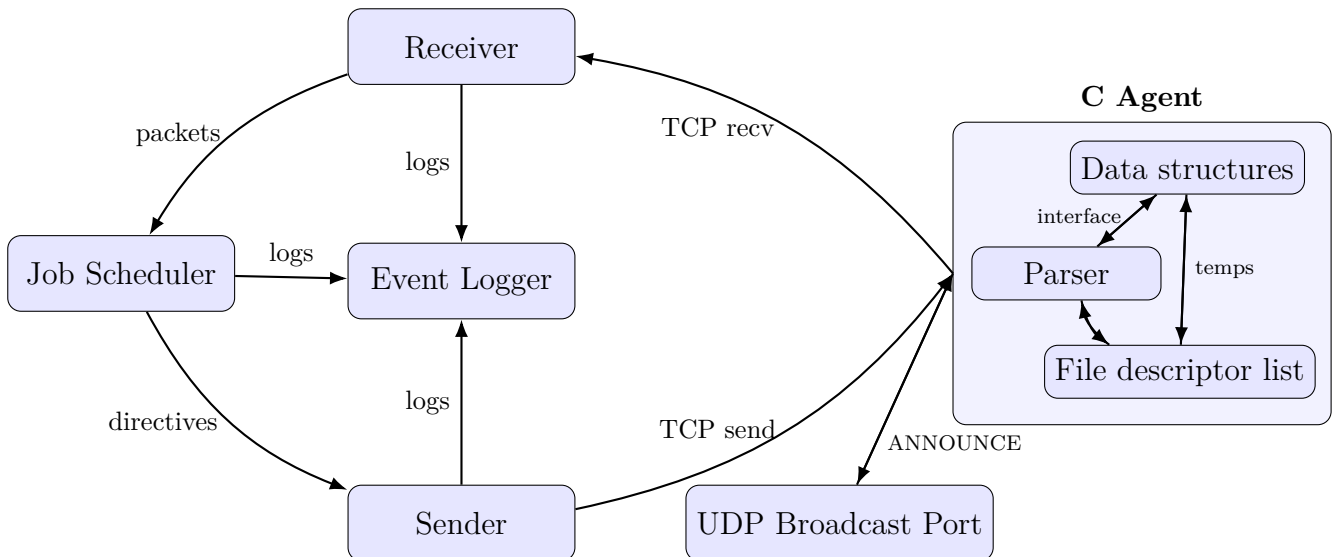


Figura 1: System communication architecture: Erlang processes and C agent internals

2.3. Convención del Entorno de Pruebas

Para simplificar las pruebas de interbloqueo sin las restricciones del ordenamiento global de recursos, se introdujo una convención para la variable de entorno **ENV**. Cuando **ENV** se establece en "test", la función `generate_job_request` omite el paso de ordenamiento de recursos, permitiendo que los trabajos soliciten recursos en un orden arbitrario. Esto hace posible activar deliberadamente el escenario de interbloqueo descrito en la Sección 4 y verificar que el sistema lo detecte o se recupere de él.

En producción (**ENV** sin establecer o con cualquier otro valor), siempre se aplica el ordenamiento global y el interbloqueo se previene estructuralmente.

2.4. Secuencia de Inicio

2.5. Secuencia de Inicio del Agente C

El agente C se inicia con cinco argumentos: dirección IP, puerto TCP y los recursos disponibles de CPU, RAM y GPU. La secuencia de inicio es la siguiente:

1. El **ServerContext** se inicializa con los argumentos proporcionados y se crea una nueva instancia de `epoll`.
2. Se abren tres sockets: un listener TCP público vinculado a la IP de la LAN del nodo, un listener TCP local vinculado exclusivamente a 127.0.0.1 para el planificador Erlang, y un socket UDP para el descubrimiento por difusión (broadcast).
3. Se configuran tres instancias de `timerfd`: un temporizador TCP de un solo disparo (2 segundos), un temporizador periódico de anuncios UDP (cada 1 segundo) y un temporizador periódico del recolector de basura (cada 5 segundos).
4. Los tres temporizadores y el socket UDP se registran en `epoll`. Los sockets TCP intencionalmente **no** se registran todavía.
5. Se inicializa un pool de hilos con 4 hilos de trabajo que comienzan a esperar tareas.

6. Comienza el bucle principal de epoll. Durante los primeros 2 segundos, solo se procesa tráfico UDP: el nodo difunde su presencia y recopila anuncios de pares que ya están en la red.
7. Cuando expira el temporizador TCP, ambos sockets TCP se añaden a epoll y el agente comienza a aceptar conexiones de otros agentes y del planificador Erlang local.

Cuando se llama a `erlang_c_bridge:init/0`:

1. Genera (spawns) el proceso `event_logger` y lo registra bajo el átomo `eventLoggerId`.
2. Genera el proceso `job_scheduler`.
3. Llama a `connect/3`, que intenta hasta `?TRIES` conexiones TCP al agente C.
4. En caso de éxito, genera `conn_handler`, el cual genera a su vez `sender` y `receiver`, y notifica al planificador con sus PIDs.
5. Al agotarse los reintentos, registra el fallo en el log y llama a `erlang:halt/0`.

3. Protocolo de Comunicación

3.1. Protocolo Inter-Agente (C a C)

Los agentes C se comunican directamente entre sí a través de TCP utilizando un protocolo ASCII orientado a líneas, donde cada mensaje termina con `\n`. Este protocolo se utiliza para reservar y liberar recursos en nodos remotos.

Mensajes enviados entre agentes:

```
1 RESERVE <job_id> <resource_name> <amount>
2 GRANTED <job_id>
3 DENIED <job_id>
4 RELEASE <job_id> <resource_name> <amount>
```

Donde `job_id` es un entero único generado por el planificador Erlang, `resource_name` es uno de `cpu`, `mem` o `gpu`, y `amount` es la cantidad solicitada. El descubrimiento de nodos se maneja mediante difusión (broadcast) UDP. Cada agente anuncia periódicamente su presencia con:

```
1 ANNOUNCE <port> <cpu:N> <mem:N> <gpu:N>
```

La IP del remitente se extrae directamente de `recvfrom()` en lugar de incluirse en el mensaje. Si no se recibe ningún anuncio de un nodo dentro de los 15 segundos, se considera muerto y se elimina de la tabla de nodos conocidos.

3.2. Interfaz Local (Erlang a Agente C)

El proceso `sender` reenvía las directivas del planificador al agente C:

```
1 GET_NODES
2 JOB_REQUEST <job_id> @<ip>:<port>:<resource>:<amount> ...
3 JOB_RELEASE <job_id>
```

El proceso `receiver` escucha en el mismo socket y reenvía las respuestas al planificador:

```
1 NODES <ip>:<port>:<res>:<amount>:...;<ip>:...  
2 JOB_GRANTED <job_id>  
3 JOB_DENIED <job_id>  
4 JOB_TIMEOUT <job_id>
```

3.3. Diagrama de Secuencia: Ciclo de Vida de un Trabajo

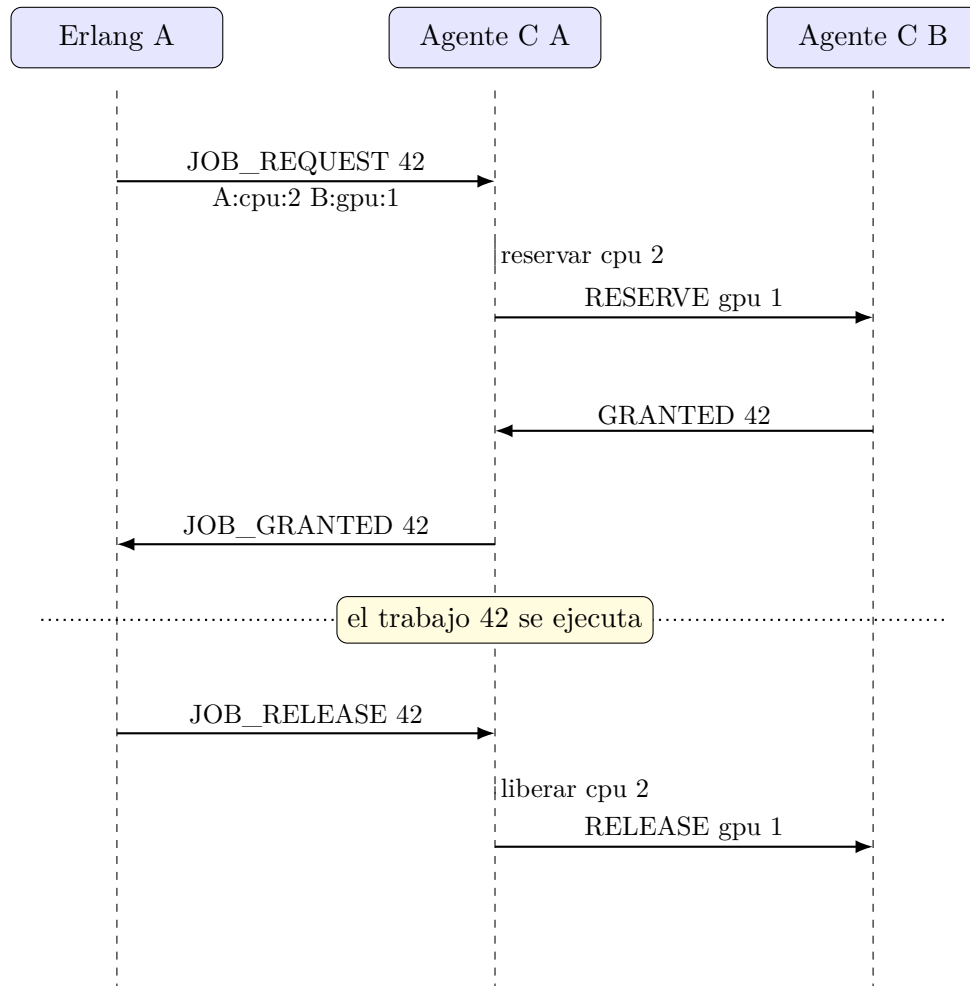


Figura 2: Ejecución y liberación exitosa de un trabajo.

4. Estrategia de Prevención de Deadlocks

4.1. Descripción del Problema

El deadlock distribuido ocurre cuando dos trabajos retienen cada uno un recurso que el otro necesita, formando una espera circular. El escenario clásico de la consigna:

- **Job1** (del Nodo A): necesita 2 CPUs del Nodo A y 1 GPU del Nodo B.
- **Job2** (del Nodo B): necesita 1 GPU del Nodo B y 2 CPUs del Nodo A.

Si ambos trabajos adquieren su primer recurso simultáneamente, cada uno se bloquea esperando el recurso retenido por el otro, lo que genera un deadlock.

4.2. Estrategia: Ordenamiento Global de Recursos

La estrategia elegida elimina la condición de **espera circular** (condición 4 de Coffman) al imponer un ordenamiento global en todas las solicitudes de recursos antes de enviarlas. Antes de enviar un `JOB_REQUEST`, el planificador ordena la lista de recursos requeridos por el par `(NodeIp, ResourceType)` en orden ascendente. El orden de los tipos se define de forma centralizada en `header.hrl`:

```

1 -define (RES_ORDER (Type) ,
2     case Type of
3         cpu -> 1;
4         mem -> 2;
5         gpu -> 3
6     end
7 ).

```

El ordenamiento se aplica en `generate_job_request/2`:

```

1 SortedResources = lists:sort(
2     fun ({Ip1, Resource1, _}, {Ip2, Resource2, _}) ->
3         {Ip1, Resource1} =< {Ip2, Resource2}
4     end,
5     AvailableResources
6 ),

```

4.3. Por Qué Funciona: Ejemplo Paso a Paso

Con el invariante de ordenamiento aplicado, el escenario de deadlock se resuelve de la siguiente manera:

1. Tanto el Trabajo1 como el Trabajo2 ordenan sus listas de recursos. Dado que $A < B$ lexicográficamente, ambas listas comienzan con los recursos del Nodo A.
2. Ambos trabajos compiten primero por las CPUs del Nodo A. Uno gana; el otro espera en la cola FIFO del Nodo A.
3. El trabajo ganador (Trabajo1) luego solicita la GPU del Nodo B y la adquiere.
4. El Trabajo1 finaliza y libera ambos recursos. El Trabajo2 luego adquiere las CPUs del Nodo A, luego la GPU del Nodo B y se completa normalmente.

El invariante clave es: *ningún trabajo puede retener un recurso de un nodo “posterior” mientras espera un recurso de un nodo “anterior”*. La espera circular es estructuralmente imposible.

5. Registro de Eventos (Event Logger)

5.1. Diseño

El logger se ejecuta como un proceso Erlang dedicado, registrado bajo el átomo `eventLoggerId`. Cualquier módulo registra un evento llamando a:

```
1 event_logger:log_event(Status, SrcMethod, Detail, JobInvolved)
```

Esto construye un registro `#logInfo{}` y lo envía como un mensaje al logger, que lo escribe en `event_logger.txt` en modo de adición (append) utilizando I/O de archivos nativa (`raw`).

5.2. Registro de Log (Log Record)

```
1 -record(logInfo, {
2     status,           % ok / error / close
3     src_method,       % {Modulo, Funcion}
4     detail,           % atom o string que describe el evento
5     job_involved,     % entero job_id / none
6     timestamp        % {Hora, Min, Seg}
7 }).
```

5.3. Ejemplo de Salida

```
1 ok {erlang_c_bridge,init} initialization none {14,32,10}
2 ok {erlang_c_bridge,connect} "CONNECTION SUCCESSFUL" none {14,32,10}
3 ok {job_scheduler,init} "[job_scheduler] sender_pid received" none
  {14,32,11}
4 ok {job_scheduler,init} "[job_scheduler] job_request" 5 {14,32,16}
5 ok {job_scheduler,init} "[job_scheduler] job_granted" 5 {14,32,16}
```

5.4. Justificación del Diseño

Registrar el logger bajo un átomo fijo evita tener que pasar el PID del logger a través de cada llamada a función. Cualquier proceso en el sistema puede registrar un evento en cualquier momento sin necesidad de mantener una referencia.

6. Contribuciones del Equipo

Miembro	Contribuciones
[Lucio Oviedo Dell'Orefice]	Agente C: servidor epoll, manejo de TCP, E/S no bloqueante.
[Juan Ignacio Gorgo]	Agente C: gestión de recursos, colas FIFO, lógica GRANTED/DENIED.
[Camila Sarasúa]	Erlang: planificador de trabajos, prevención de interbloqueos, máquina de estados de los trabajos.
[Santiago Salvático]	Erlang: <code>erlang_c_bridge</code> , <code>event_logger</code> , <code>header.hrl</code> , documentación, integración.

7. Desafíos de Implementación

Durante el desarrollo del gestor de recursos distribuido, se identificaron y resolvieron varios problemas de implementación. Estos problemas influyeron tanto en el protocolo de comunicación como en la arquitectura interna del sistema.

7.1. Identificación de Agentes

Inicialmente, los agentes C se identificaban únicamente por su dirección IP. Este enfoque resultó insuficiente cuando se ejecutaban múltiples agentes en el mismo host, ya que diferentes nodos podían compartir la misma IP mientras escuchaban en puertos distintos. Para resolver este problema, los agentes se identificaron de forma única utilizando el par:

(IP, Puerto)

Este identificador pasó a formar parte del descubrimiento de nodos, la comunicación remota y los mensajes de asignación de recursos, lo que permitió que múltiples agentes coexistieran en la misma máquina.

7.2. Propiedad de Recursos Remotos

A medida que el sistema evolucionó para admitir múltiples agentes, los recursos podían pertenecer a diferentes nodos. Una solicitud de trabajo podría requerir recursos distribuidos en varias máquinas. Por esta razón, los descriptores de recursos se ampliaron para incluir la información del nodo de destino. Por lo tanto, las operaciones de reserva y liberación se dirigen al agente remoto correcto, lo que garantiza que los recursos se asignen y liberen de manera consistente.

7.3. Interoperabilidad del Parser entre C y Erlang

La comunicación entre el agente C y el planificador Erlang expuso varios problemas de interoperabilidad. Inicialmente, algunos mensajes generados por el agente C contenían caracteres de nueva línea al final. Cuando Erlang intentaba convertir valores como:

1 "1\n"

directamente en enteros, se producían fallos de análisis (parsing). El parser de Erlang se modificó para normalizar los campos entrantes usando `string:trim/1` antes de convertir los valores, mejorando la robustez frente a diferencias de formato.

7.4. Tráfico de Descubrimiento UDP

El mecanismo de descubrimiento distribuido se basa en anuncios periódicos de UDP entre agentes. Las primeras versiones utilizaban un intervalo de anuncio muy corto, lo que producía un tráfico de red innecesario y una salida de log excesiva. Además, los parámetros de inicio incorrectos ocasionalmente causaban mensajes de anuncio no válidos. El protocolo de descubrimiento se estabilizó aumentando el intervalo de difusión (broadcast) y estandarizando el formato del anuncio:

```
1 ANNOUNCE <port> cpu:<cpu> mem:<memory> gpu:<gpu>
```

Esto redujo el ruido de la red manteniendo un descubrimiento de nodos confiable.

8. Problemas Encontrados y Soluciones

8.1. Colisión de Job ID entre Nodos en la Misma IP

Problema: La asignación específica que dos nodos pueden compartir la misma dirección IP siempre que utilicen puertos diferentes. Dado que el programador (scheduler) de Erlang de cada nodo genera IDs de trabajo de forma independiente utilizando `erlang:unique_integer/1`, dos programadores en la misma IP pueden producir IDs de trabajo idénticos. Cuando un tercer agente recibe un mensaje `RESERVE 42 cpu 2`, no puede determinar qué nodo lo envió basándose únicamente en el ID de trabajo.

Solución: El agente en C ya tiene acceso al descriptor de archivo (`fd`) de la conexión TCP entrante a través del parámetro `SOCKET` en `resource_adapter_patch`. Dado que el kernel garantiza que no hay dos conexiones activas que compartan el mismo `fd` dentro de un proceso, el par (`job_id`, `fd`) es globalmente único:

- El `fd` identifica de forma única qué nodo envió el mensaje.
- Dentro de la misma conexión (`fd`), `erlang:unique_integer` garantiza que el ID de trabajo nunca se repita.

La solución no requirió cambios en el protocolo de red: el `fd` ya está disponible en toda la tubería de manejo de eventos y solo fue necesario incorporarlo como parte de la clave de trabajo en la tabla de trabajos activos.

8.2. Cierre prematuro del socket

Problema: Cuando el proceso que abrió un socket TCP finaliza, Erlang cierra el socket automáticamente. Esto provocaba que la conexión se cayera cuando `conn_handler` generaba procesos hijos y luego terminaba.

Solución: El ID del socket se pasa directamente a los procesos `sender` (emisor) y `receiver` (receptor) en el momento de su creación. Dado que estos son procesos de larga duración, el socket permanece abierto durante toda la sesión.

8.3. Registro del ID del proceso de registro (Logger)

Problema: Pasar el PID del logger a través de cada llamada a función creaba ruido innecesario entre los módulos.

Solución: El logger se registra bajo el átomo `eventLoggerId` mediante `register/2`. Todos los módulos envían mensajes de registro directamente al átomo.

8.4. Bloqueo de lectura TCP vs. paso de mensajes en Erlang

Problema: Un proceso bloqueado en `gen_tcp:recv` con `active, false` no puede recibir mensajes de Erlang simultáneamente. La alternativa, `active, true`, entrega los datos TCP directamente al buzón del proceso; pero si el agente C envía mensajes más rápido de lo que el proceso los consume, el buzón puede agotar la memoria.

Solución: Mantuvimos `active, false` y separamos el tráfico entrante y saliente en dos procesos dedicados:

- `receiver`: se bloquea en `gen_tcp:recv` esperando las respuestas del agente C.
- `sender`: se bloquea en `receive` esperando las directivas del programador.

Cada proceso tiene una única responsabilidad de bloqueo, evitando ambos problemas.